1. University Scholarship Allocation

Problem Statement

Create a program that checks whether a student is eligible for a scholarship using aggregate marks, attendance, and family income.

Objectives

• Accept student marks, attendance, and annual family income.

• Check the given eligibility conditions and display the result.

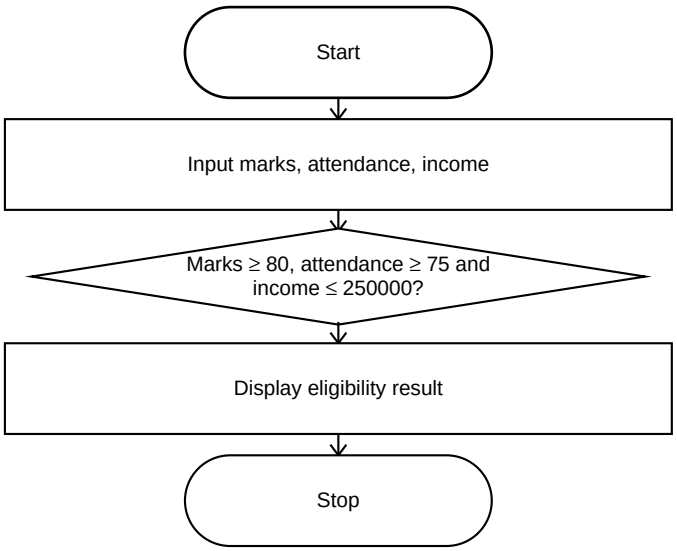
Assumptions

A student is eligible when marks are at least 80%, attendance is at least 75%, and annual family income is at most ■250000.

Algorithm

1. Start.
2. Read marks, attendance, and income.
3. If all three conditions are satisfied, display Eligible; otherwise display Not eligible.
4. Stop.

Flowchart



Pseudocode

```
BEGIN
  INPUT marks, attendance, income
  IF marks >= 80 AND attendance >= 75 AND income <= 250000 THEN
      PRINT "Eligible for scholarship"
  ELSE
      PRINT "Not eligible"
  END IF
END
```

Python Source Code

```
marks = float(input("Enter aggregate marks (%): "))
attendance = float(input("Enter attendance (%): "))
income = float(input("Enter annual family income (Rs): "))
if marks >= 80 and attendance >= 75 and income <= 250000:
    print("Eligible for scholarship")
else:
    print("Not eligible")
```

Test Cases / Sample Output

Input: marks=85, attendance=90, income=200000 → Output: Eligible for scholarship

Input: marks=78, attendance=90, income=200000 → Output: Not eligible

Conclusion

The program applies multiple conditions to decide scholarship eligibility.

2. Progressive Electricity Billing

Problem Statement

Calculate an electricity bill using different consumption slabs and a fixed charge.

Objectives

• Read units consumed.

• Calculate the energy charge using progressive slabs and add a fixed charge.

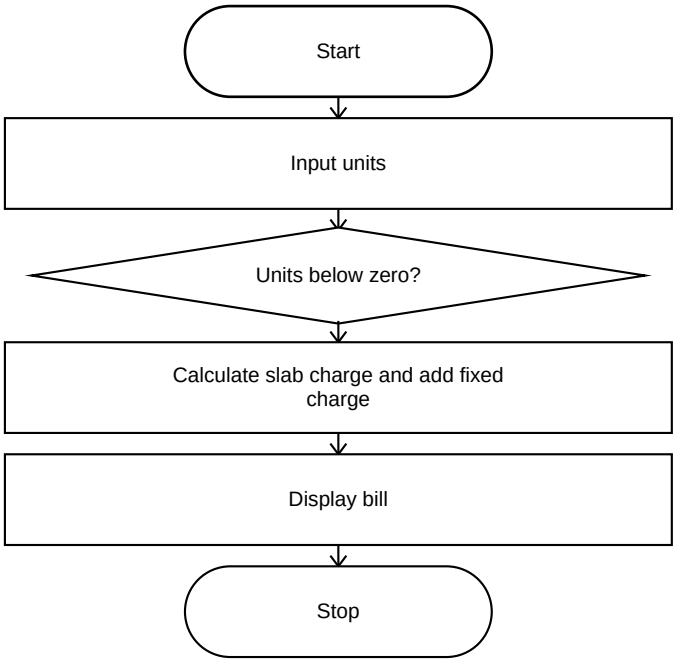
Assumptions

First 100 units cost I2/unit, next 100 units cost ■3/unit, and units above 200 cost I5/unit. Fixed charge is ■50.

Algorithm

1. Start and read units.
2. Reject negative units.
3. Calculate each slab charge.
4. Add ■50 fixed charge and display the bill.
5. Stop.

Flowchart



Pseudocode

```
BEGIN
  INPUT units
  IF units < 0 THEN PRINT "Invalid units"
  ELSE
    IF units <= 100 THEN energy = units * 2
    ELSE IF units <= 200 THEN energy = 100*2 + (units-100)*3
    ELSE energy = 100*2 + 100*3 + (units-200)*5
    bill = energy + 50
    PRINT bill
  END IF
END
```

Python Source Code

```
units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5
    bill = energy_charge + 50
    print("Electricity bill: Rs.", bill)
```

Test Cases / Sample Output

Input: 80 → Output: Electricity bill: Rs. 210

Input: 150 → Output: Electricity bill: Rs. 300

Input: 250 → Output: Electricity bill: Rs. 650

Conclusion

The program calculates bills correctly using progressive consumption slabs.

3. Hospital Billing and Patient Classification

Problem Statement

Calculate a patient's hospital bill from consultation, diagnostic, and treatment charges, and apply a concession when eligible.

Objectives

• Add the charges to find the total.

• Apply a concession based on the stated rule.

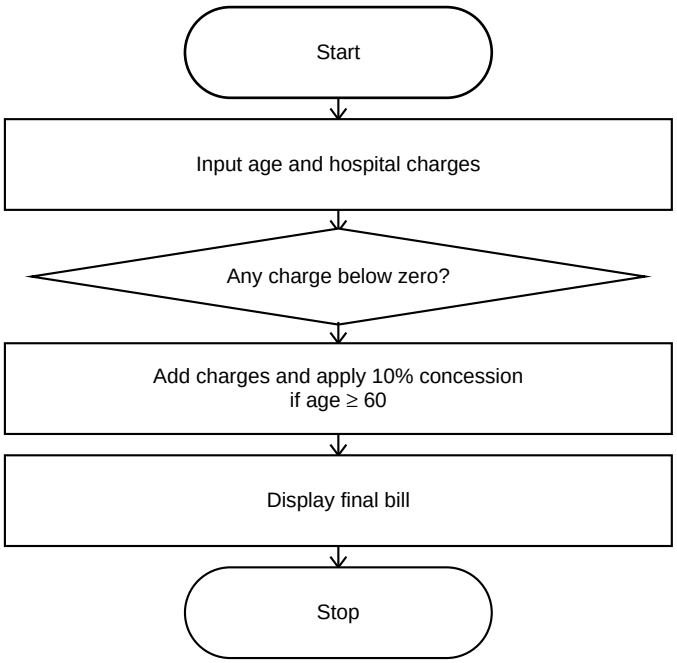
Assumptions

Patients aged 60 or above receive a 10% concession. All charges must be zero or positive.

Algorithm

1. Read age and three charges.
2. Check that charges are non-negative.
3. Add charges to calculate total.
4. If age is at least 60, reduce total by 10%.
5. Display final bill.

Flowchart



Pseudocode

```
BEGIN
  INPUT age, consultation, diagnostic, treatment
  IF any charge < 0 THEN PRINT "Invalid charge"
  ELSE
    total = consultation + diagnostic + treatment
    IF age >= 60 THEN total = total * 0.90
    PRINT total
  END IF
END
```

Python Source Code

```
age = int(input("Enter patient age: "))
consultation = float(input("Consultation charge: Rs. "))
diagnostic = float(input("Diagnostic charge: Rs. "))
treatment = float(input("Treatment charge: Rs. "))
if consultation < 0 or diagnostic < 0 or treatment < 0:
    print("Invalid charge")
else:
    total = consultation + diagnostic + treatment
    if age >= 60:
        total = total * 0.90
        print("10% senior concession applied")
    print("Final hospital bill: Rs.", round(total, 2))
```

Test Cases / Sample Output

Input: age=65, charges=500, 1000, 2500 → Output: Final hospital bill: Rs. 3600.0

Input: age=30, charges=500, 1000, 2500 → Output: Final hospital bill: Rs. 4000.0

Conclusion

The program totals hospital charges and applies the specified senior-patient concession.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate an employee's gross salary from basic pay, allowance, overtime, and incentive, then subtract deductions.

Objectives

• Calculate gross salary.

• Calculate deductions and net salary.

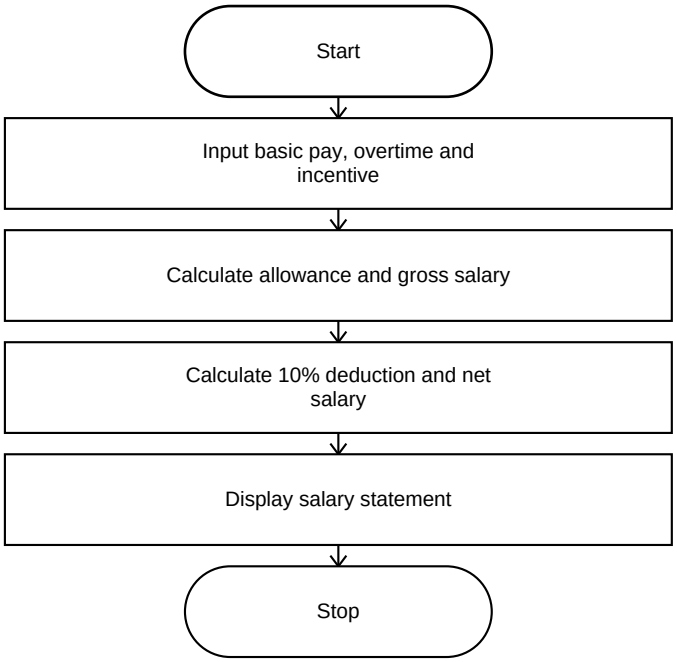
Assumptions

Allowance is 20% of basic pay, overtime and incentive are entered by the user, and deduction is 10% of gross salary.

Algorithm

1. Read basic pay, overtime, and incentive.
2. Calculate allowance and gross salary.
3. Calculate deduction as 10% of gross salary.
4. Calculate net salary and display the statement.

Flowchart



Pseudocode

```
BEGIN
  INPUT basic, overtime, incentive
  allowance = basic * 0.20
  gross = basic + allowance + overtime + incentive
  deduction = gross * 0.10
  net = gross - deduction
  PRINT gross, deduction, net
END
```

Python Source Code

```
basic = float(input("Enter basic pay: Rs. "))
overtime = float(input("Enter overtime pay: Rs. "))
incentive = float(input("Enter incentive: Rs. "))
allowance = basic * 0.20
gross = basic + allowance + overtime + incentive
deduction = gross * 0.10
net_salary = gross - deduction
print("Allowance: Rs.", round(allowance, 2))
print("Gross salary: Rs.", round(gross, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net_salary, 2))
```

Test Cases / Sample Output

Input: basic=20000, overtime=1000, incentive=500 → Output: allowance=4000, gross=25500, deduction=2550, net salary=2295

Conclusion

The program generates a simple salary statement using arithmetic operations.

5. Mobile Plan Cost Comparison

Problem Statement

Compare three mobile plans and identify the least expensive plan that satisfies the user's data requirement and budget.

Objectives

• Store plan details in lists.

• Check data and budget requirements.

• Display the cheapest suitable plan.

Assumptions

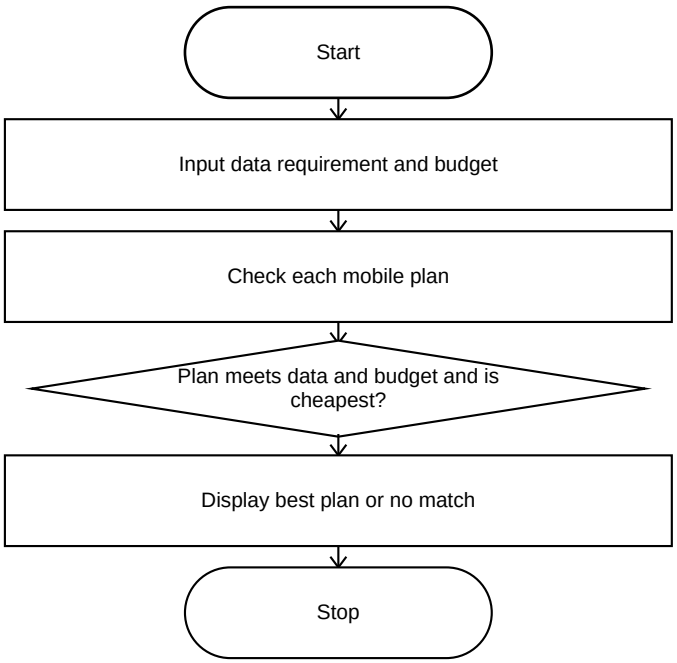
Plan details are fixed in the program. A plan is suitable if its data allowance meets the required data and its price is within

budget.

Algorithm

1. Read required data and budget.
2. Store plan names, data allowances, and prices in lists.
3. Check each plan using a loop.
4. Track the lowest-priced suitable plan.
5. Display the plan or report no match.

Flowchart



Pseudocode

```
BEGIN
  INPUT required_data, budget
  plans = ["Basic", "Plus", "Max"]
  data = [2, 5, 10]
  price = [199, 299, 399]
  best = -1
  FOR each plan index i
    IF data[i] >= required_data AND price[i] <= budget THEN
      IF best == -1 OR price[i] < price[best] THEN best = i
  IF best == -1 THEN PRINT "No suitable plan"
  ELSE PRINT plans[best], price[best]
END
```

Python Source Code

```
required_data = float(input("Required data (GB): "))
budget = float(input("Your budget (Rs.): "))
plans = ["Basic", "Plus", "Max"]
data_gb = [2, 5, 10]
prices = [199, 299, 399]
best = -1
for i in range(len(plans)):
    if data_gb[i] >= required_data and prices[i] <= budget:
        if best == -1 or prices[i] < prices[best]:
            best = i
if best == -1:
    print("No suitable plan found")
else:
    print("Best plan:", plans[best])
    print("Price: Rs.", prices[best])
```

Test Cases / Sample Output

Input: required data=4 GB, budget=350 → Output: Best plan: Plus; Price: Rs. 299

Input: required data=12 GB, budget=350 → Output: No suitable plan found

Conclusion

The program uses lists and loops to find the most economical suitable plan.

6. Railway Fare Calculator

Problem Statement

Calculate the fare for a passenger based on travel distance, class, and age concession.

Objectives

• Calculate fare from distance and class.

• Apply a concession for senior passengers.

Assumptions

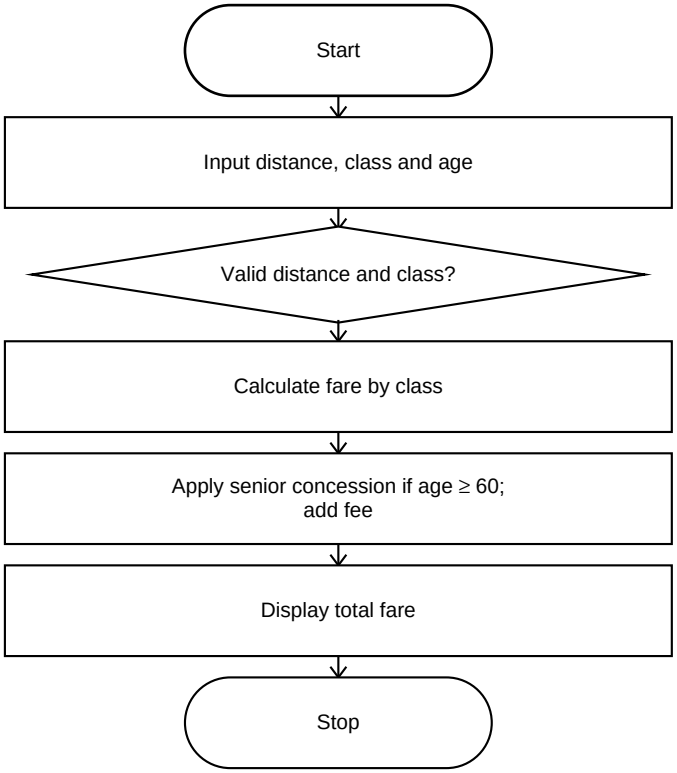
Base fare is ■1.50 per km for Sleeper and ■3 per km for AC. Passengers aged 60 or above receive a 20% concession.

Reservation fee is ■40.

Algorithm

1. Read distance, class, and age.
2. Validate distance.
3. Calculate base fare according to class.
4. Apply age concession when applicable.
5. Add reservation fee and display fare.

Flowchart



Pseudocode

```
BEGIN
  INPUT distance, class, age
  IF distance <= 0 THEN PRINT "Invalid distance"
  ELSE
    IF class == "Sleeper" THEN fare = distance * 1.5
    ELSE IF class == "AC" THEN fare = distance * 3
    ELSE PRINT "Invalid class"
    IF age >= 60 THEN fare = fare * 0.80
    fare = fare + 40
    PRINT fare
  END IF
END
```

Python Source Code

```
distance = float(input("Enter distance in km: "))
travel_class = input("Enter class (Sleeper/AC): ").strip().lower()
age = int(input("Enter passenger age: "))
if distance <= 0:
    print("Invalid distance")
elif travel_class not in ["sleeper", "ac"]:
    print("Invalid class")
else:
    if travel_class == "sleeper":
        fare = distance * 1.5
    else:
        fare = distance * 3
    if age >= 60:
        fare = fare * 0.80
    fare = fare + 40
    print("Total fare: Rs.", round(fare, 2))
```

Test Cases / Sample Output

Input: distance=100, class=Sleeper, age=30 → Output: Total fare: Rs. 190.0

Input: distance=100, class=AC, age=65 → Output: Total fare: Rs. 280.0

Conclusion

The program calculates a sample railway fare with class-based rates and a concession.

7. Retail Billing with Discount

Problem Statement

Prepare a retail bill using item prices and quantities, and apply a discount when the subtotal reaches a specified amount.

Objectives

• Calculate subtotal for several products.

• Apply a threshold discount and display the final bill.

Assumptions

Three products are entered. A 10% discount is applied when the subtotal is at least ■2000. Tax is not included in this simplified example.

Algorithm

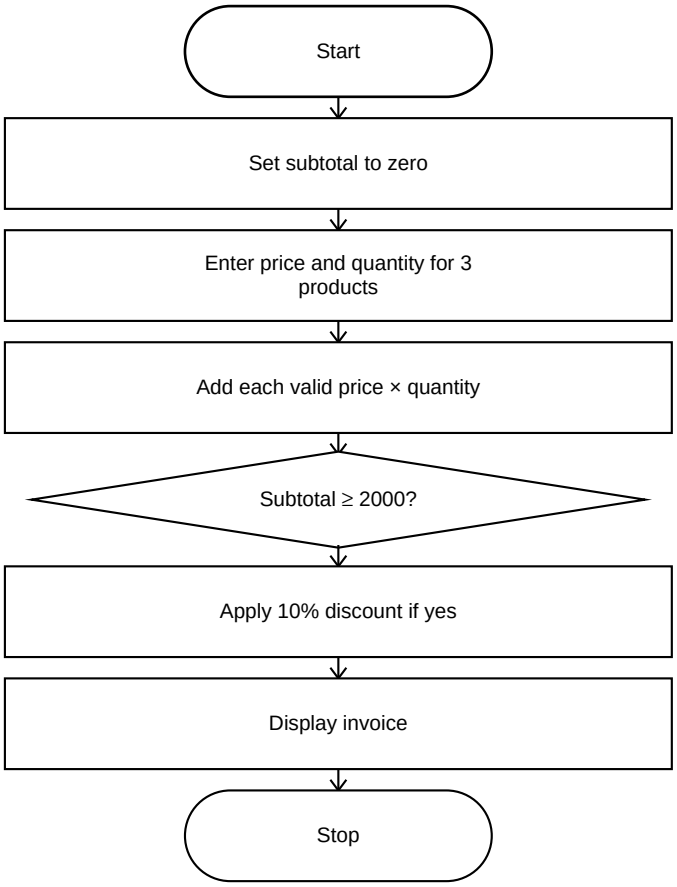
1. Set subtotal to zero.

2. Repeat for three products: read price and quantity, validate them, and add line total.

3. Apply 10% discount if subtotal is at least ■2000.

4. Display subtotal, discount, and final bill.

Flowchart



Pseudocode

```
BEGIN
  subtotal = 0
  REPEAT 3 TIMES
    INPUT price, quantity
    IF price < 0 OR quantity < 0 THEN PRINT "Invalid item"
    ELSE subtotal = subtotal + price * quantity
  IF subtotal >= 2000 THEN discount = subtotal * 0.10
  ELSE discount = 0
  PRINT subtotal, discount, subtotal - discount
END
```

Python Source Code

```
subtotal = 0
for i in range(3):
    print("Product", i + 1)
    price = float(input("Enter unit price: Rs. "))
    quantity = int(input("Enter quantity: "))
    if price < 0 or quantity < 0:
        print("Invalid item values")
    else:
        subtotal = subtotal + price * quantity
if subtotal >= 2000:
    discount = subtotal * 0.10
else:
    discount = 0
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Final bill: Rs.", round(subtotal - discount, 2))
```

Test Cases / Sample Output

Items: ■1000 × 1, ■600 × 1, ■500 × 1 → Subtotal ■2100, discount ■210, final bill ■1890

Conclusion

The program calculates a multi-item bill and applies a discount based on the subtotal.

8. Manufacturing Cost and Profit Analysis

Problem Statement

Calculate the total manufacturing cost and determine whether the product meets a target profit.

Objectives

• Add material, labour, and machine costs.

• Calculate profit and compare it with a target.

Assumptions

Total cost is the sum of material, labour, and machine costs. Profit equals selling revenue minus total cost. The target profit

is entered by the user.

Algorithm

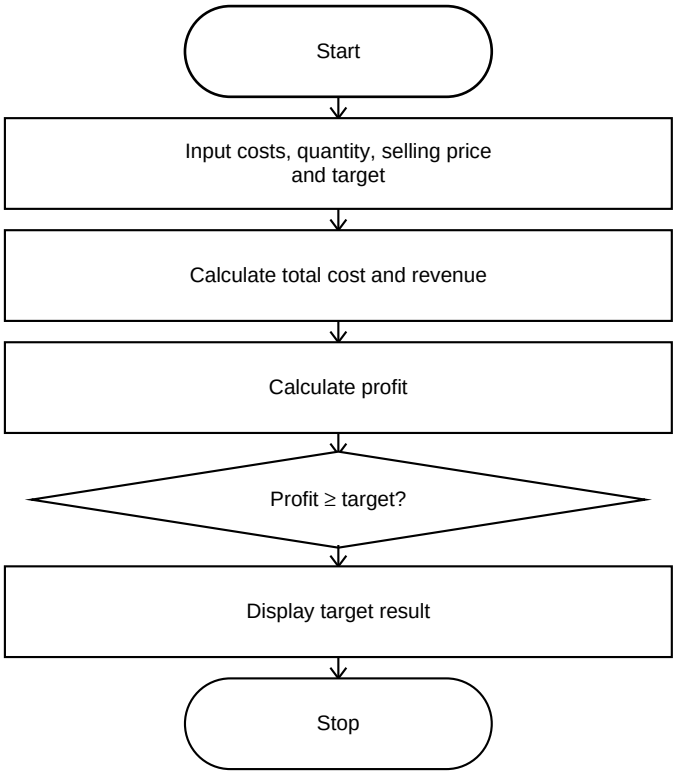
1. Read costs, quantity, selling price per unit, and target profit.

2. Calculate total cost and revenue.

3. Calculate profit.

4. Compare profit with target and display result.

Flowchart



Pseudocode

```
BEGIN
  INPUT material, labour, machine, quantity, selling_price, target
  total_cost = material + labour + machine
  revenue = quantity * selling_price
  profit = revenue - total_cost
  PRINT total_cost, revenue, profit
  IF profit >= target THEN PRINT "Target achieved"
  ELSE PRINT "Target not achieved"
END
```

Python Source Code

```
material = float(input("Material cost: Rs. "))
labour = float(input("Labour cost: Rs. "))
machine = float(input("Machine cost: Rs. "))
quantity = int(input("Quantity produced: "))
selling_price = float(input("Selling price per unit: Rs. "))
target = float(input("Target profit: Rs. "))
total_cost = material + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost
print("Total cost: Rs.", round(total_cost, 2))
print("Revenue: Rs.", round(revenue, 2))
print("Profit: Rs.", round(profit, 2))
if profit >= target:
    print("Target achieved")
else:
    print("Target not achieved")
```

Test Cases / Sample Output

Input: material=5000, labour=2000, machine=1000, quantity=100, price=100, target=1500 → cost ■8000, revenue ■10000, pr

Conclusion

The program compares production profit with a required target.

9. Multi-Currency Travel Budget

Problem Statement

Convert travel expenses in a foreign currency to rupees, include a conversion fee, and compare the total with a budget.

Objectives

• Convert expenses using an exchange rate.

• Add a conversion fee and check the budget.

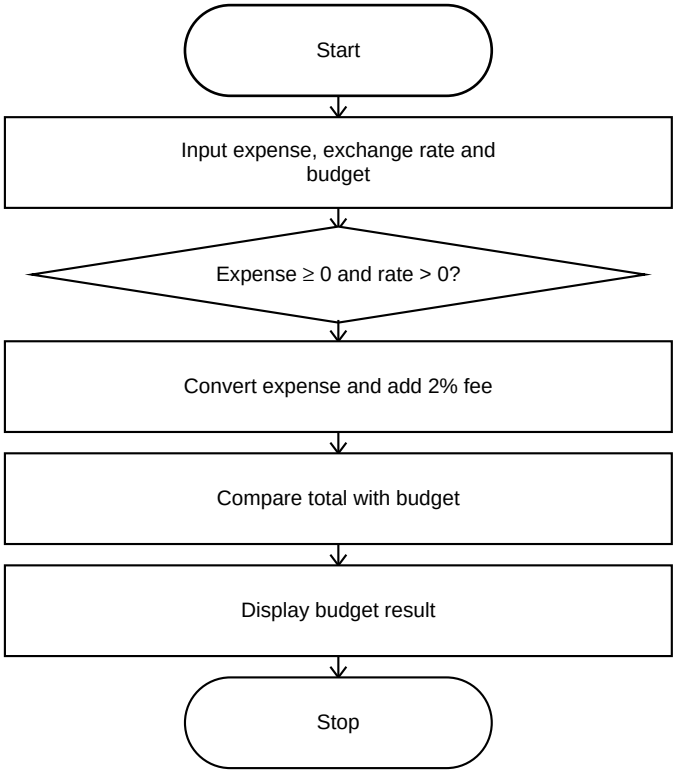
Assumptions

The user enters the exchange rate in rupees per foreign-currency unit. A 2% conversion fee is added to the converted expense.

Algorithm

1. Read foreign expense, exchange rate, and rupee budget.
2. Validate that expense and rate are non-negative and rate is positive.
3. Convert expense to rupees.
4. Add a 2% fee.
5. Compare with budget and display result.

Flowchart



Pseudocode

```
BEGIN
  INPUT expense, rate, budget
  IF expense < 0 OR rate <= 0 THEN PRINT "Invalid input"
  ELSE
    converted = expense * rate
    fee = converted * 0.02
    total = converted + fee
    PRINT total
    IF total <= budget THEN PRINT "Within budget"
    ELSE PRINT "Over budget"
  END IF
END
```

Python Source Code

```
expense = float(input("Expense in foreign currency: "))
rate = float(input("Exchange rate (Rs. per unit): "))
budget = float(input("Travel budget in Rs.: "))
if expense < 0 or rate <= 0:
    print("Invalid input")
else:
    converted = expense * rate
    fee = converted * 0.02
    total = converted + fee
    print("Converted amount: Rs.", round(converted, 2))
    print("Conversion fee: Rs.", round(fee, 2))
    print("Total expense: Rs.", round(total, 2))
    if total <= budget:
        print("Within budget")
    else:
        print("Over budget")
```

Test Cases / Sample Output

Input: expense=100, rate=83, budget=9000 → converted ■8300, fee ■166, total ■8466, Within budget

Conclusion

The program converts travel expenses and checks whether spending fits the budget.

10. Campus Event Financial Planning

Problem Statement

Calculate the total cost of a campus event and find the minimum number of registrations needed to cover the cost after sponsorship.

Objectives

• Calculate total event expenses.

• Find the remaining amount after sponsorship.

• Calculate the minimum registrations needed to break even.

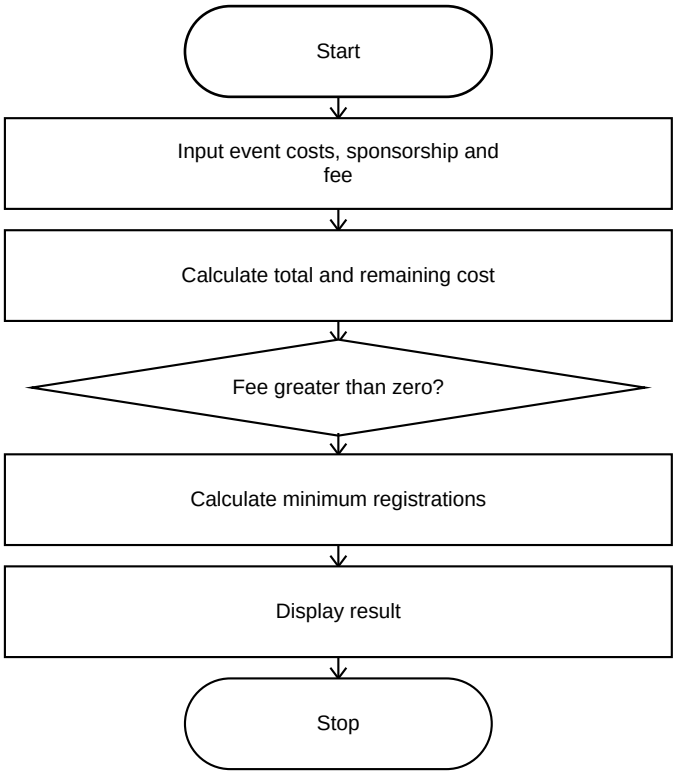
Assumptions

Participant fee is fixed. Any fraction of a participant is rounded up because registrations must be whole numbers.

Algorithm

1. Read venue, equipment, catering, transport, certificate costs, sponsorship, and fee per participant.
2. Calculate total cost.
3. Subtract sponsorship to find the remaining cost.
4. If fee is positive, calculate registrations using ceiling-style integer arithmetic.
5. Display the result.

Flowchart



Pseudocode

```
BEGIN
  INPUT venue, equipment, catering, transport, certificates, sponsorship, fee
  total = venue + equipment + catering + transport + certificates
  remaining = total - sponsorship
  IF remaining < 0 THEN remaining = 0
  IF fee <= 0 THEN PRINT "Invalid participant fee"
  ELSE registrations = (remaining + fee - 1) // fee
       PRINT total, remaining, registrations
END
```

Python Source Code

```
venue = float(input("Venue cost: Rs. "))
equipment = float(input("Equipment cost: Rs. "))
catering = float(input("Catering cost: Rs. "))
transport = float(input("Transport cost: Rs. "))
certificates = float(input("Certificate cost: Rs. "))
sponsorship = float(input("Sponsorship amount: Rs. "))
fee = float(input("Registration fee per participant: Rs. "))
total_cost = venue + equipment + catering + transport + certificates
remaining = max(0, total_cost - sponsorship)
print("Total event cost: Rs.", round(total_cost, 2))
print("Amount to be covered: Rs.", round(remaining, 2))
if fee <= 0:
    print("Registration fee must be greater than zero")
else:
    registrations = int(remaining / fee)
    if registrations * fee < remaining:
        registrations += 1
    print("Minimum registrations needed:", registrations)
```

Test Cases / Sample Output

Input costs=10000, 3000, 12000, 2000, 1000; sponsorship=8000; fee=200 → total ■28000, remaining ■20000, minimum regist

Conclusion

The program estimates event finances and calculates the registrations needed to break even.